

1. Stack memory (automatic variables)

Stack frames store all automatic objects created inside main.

Each function call adds a new frame that is released automatically when the function ends.

Inside the current stack frame for main, we have

Variable	Where it lives	What it holds
forder	stack	a FlexibleOrder object containing three data members: items (pointer), capacity (integer), and count (integer)
item1	stack	an Item object whose internal members (for example, its std::string name) each manage their own heap memory for the text "banana"
item2	stack	another Item object managing heap memory for "natural peanut butter"

So the **object forder itself**—its integers and its pointer items—are all stored contiguously inside the stack frame for main.

2. Heap memory (dynamic variables)

When the FlexibleOrder constructor ran, it executed

```
items = new Item[capacity];
```

which allocated a **dynamic array of Item objects on the heap**.

Initially capacity is 5, so the heap contains space for 5 Items.

The pointer forder.items (on the stack) holds the memory address of the first Item in that array.

After calling add(item1) and add(item2):

- The first two slots of the heap array store **copies** of item1 and item2.
- The remaining three slots are default-constructed Items waiting to be used.
- The count field (on the stack) now equals 2.
- The capacity field (on the stack) remains 5.

3. Destruction sequence

When main returns, its stack frame unwinds automatically:

1. `forder`'s destructor runs.
It executes `delete[] items`; to release the 5-element heap array.
2. Each `Item` inside that array has its own destructor called automatically.
3. Then `forder`, `item1`, and `item2` themselves are removed from the stack.

This matches the RAII pattern: each object frees any resources it acquired.